Flash-SD-KDE: Accelerating SD-KDE with Tensor Cores

Elliot L. Epstein [1] **Rajat Vadiraj Dwaraknath** [1] **John Winnicki** [1]

Abstract

Score-debiased kernel density estimation (SD-KDE) achieves improved asymptotic convergence rates over classical KDE, but its use of an empirical score has made it significantly slower in practice. We show that by re-ordering the SD-KDE computation to expose matrix-multiplication structure, Tensor Cores can be used to accelerate the GPU implementation. On a 32k-sample 16-dimensional problem, our approach runs up to $47\times$ faster than a strong SD-KDE GPU baseline and $3{,}300\times$ faster than scikit-learn's KDE. On a larger 1M-sample 16-dimensional task evaluated on 131k queries, Flash-SD-KDE completes in 2.3 s on a single GPU, making score-debiased density estimation practical at previously infeasible scales. Code available at `https://github.com/Elliotepsteino/Flash-SD-KDE`.

1. Introduction

Kernel density estimation (KDE) is a foundational nonparametric tool (Rosenblatt, 1956; Parzen, 1962; Silverman, 1986; Wand & Jones, 1995) that has become a recurring primitive in modern machine learning pipelines: KDE-style density and score estimates underpin anomaly and out-of-distribution detection (Luo & Shrivastava, 2018; Coleman & Shrivastava, 2020), embedding-space dataset comparison and divergence estimation, data-mixture and selection analysis for pre-training and instruction tuning, density-based regularization and sampling, and softmax-style attention (Zandieh et al., 2023; Kacham et al., 2024). Two limitations have long held KDE back at these scales: an $O(h^2)$ leading-bias term that dominates error in moderate dimensions, and an $O(n^2)$ exact evaluation cost — both of which have driven decades of follow-up work.

A long line of work attacks the bias limitation by modifying the kernel or the bandwidth so that the leading bias term cancels. Higher-order kernel constructions (Fan & Hu, 1992; Jones & Signorini, 1997), variable-bandwidth schemes (Abramson, 1982), and diffusion-based estimators (Botev et al., 2010) are all members of this family. Score-debiased kernel density estimation (SD-KDE) (Epstein et al., 2025) is a recent addition: it uses an empirical score $\hat{s}$ to form debiased samples $x_i^{\mathrm{SD}} = x_i + \frac{h^2}{2}\,\hat{s}(x_i)$, and then evaluates a standard Gaussian KDE on these shifted samples instead of the originals, which improves statistical efficiency over standard KDE while retaining a simple, nonparametric form and preserving non-negativity. In particular, SD-KDE achieves an asymptotic mean integrated squared error (AMISE) of order $O(n^{-8/(d+8)})$, improving the convergence exponent relative to the $O(n^{-4/(d+4)})$ rate of classical KDE tuned with Silverman's rule of thumb (Silverman, 1986; Wand & Jones, 1995).

These statistical gains come with a significant computational drawback. The empirical score used for debiasing is itself a kernel sum, so it introduces an additional $O(n^2)$ pass over the data, and a naive implementation has roughly the same quadratic cost as KDE itself but with a substantially larger constant factor. Empirically, the score pass dominates end-to-end runtime, accounting for roughly 90% of the total cost in our setting, which has so far made SD-KDE impractical at the scales where downstream applications operate. The natural fix — porting the computation to GPUs — is not automatic, because the score pass does not match the GEMM shape that modern accelerators are built to execute, and the scalar work surrounding the kernel sum (norms, exponentials, atomic accumulation) limits how much generic deep-learning frameworks can help.

Our central observation is that re-ordering the SD-KDE computation exposes a hidden matrix-multiplication structure inside both the empirical-score pass and the KDE evaluation. Once that structure is made explicit, the heavy work of SD-KDE is exactly the work that Tensor Cores on modern GPUs are designed to accelerate, and the rest of the implementation reduces to streaming the matrix tiles and managing the surrounding scalar work against a mixed-precision performance budget. This is what enables SD-KDE to run at scales that were previously infeasible on a single GPU.

Contributions. We make SD-KDE practical at large scale

[1]Institute for Computational and Mathematical Engineering (ICME), Stanford University, Stanford, CA, USA. Correspondence to: Elliot L. Epstein <epsteine@stanford.edu>.

Structured Probabilistic Inference & Generative Modeling workshop at the 43^{rd} International Conference on Machine Learning, Seoul, South Korea, 2026.

through three contributions: a GEMM re-ordering that exposes Tensor-Core structure in both the score and the KDE passes, a streaming Triton implementation that avoids materializing pairwise kernel matrices, and a fused first-order surrogate (Flash-Laplace-KDE) that retains the leading bias correction without the explicit score pass. On a 32k-sample 16-D problem, Flash-SD-KDE runs roughly $40\times$ faster than a strong PyTorch SD-KDE baseline, $\sim 8\times$ faster than PyKeOps, and over three orders of magnitude faster than scikit-learn KDE.

Paper overview. Section 2 surveys related work on KDE, score estimation, fast kernel evaluation, and GPU programming. Section 3 reviews the GPU hardware features that drive the design. Section 4 derives the matrix-form SD-KDE computation and the streaming Tensor-Core kernel. Section 5 introduces the Laplace-corrected surrogate. Section 6 reports the empirical study, including performance optimizations and the oracle-density benchmark, and Section 6.2 concludes. Additional experiments and a more complete literature review appear in the appendix.

2. Related Work

Kernel density estimation. Kernel density estimation (KDE) is a classical nonparametric approach to density estimation that dates back to the foundational work of Rosenblatt (1956) and Parzen (1962); modern treatments appear in standard references such as Silverman (1986) and Wand & Jones (1995). For smooth densities, KDE attains optimal nonparametric rates under appropriate bandwidth scaling, but in practice performance is highly sensitive to bandwidth selection. Rules-of-thumb and plug-in procedures are widely used for their simplicity, but they are typically tuned to unimodal, near-Gaussian targets and can misbehave for multimodal or heavy-tailed distributions (Silverman, 1986; Botev et al., 2010). This sensitivity is particularly pronounced in moderate and high dimensions, where the bandwidth controls both statistical bias and the computational cost of evaluating kernels at scale.

Bias reduction and adaptive smoothing. A large body of work improves KDE accuracy by reducing leading-order bias or adapting the effective smoothing scale across the sample space. One line of work constructs higher-order bias corrections by modifying kernels or combining estimators so that the dominant $O(h^2)$ bias term cancels, yielding $O(h^4)$ bias under additional smoothness assumptions (Fan & Hu, 1992; Jones & Signorini, 1997). Another class of methods uses variable bandwidths that shrink in regions of high density and expand in the tails, improving adaptivity without committing to a parametric form (Abramson, 1982; Silverman, 1986). Complementary to these ideas, diffusion-based estimators view density estimation through the lens of smoothing by a linear diffusion process, providing both adaptivity and a data-driven bandwidth selection mechanism (Botev et al., 2010). Score-debiased KDE (SD-KDE) (Epstein et al., 2025) fits into this landscape by using score information to correct the leading bias term while retaining a simple kernel estimator structure. Our Laplace-corrected formulation can be seen as an explicit higher-order correction that removes the need to form an empirical score at evaluation time, trading additional kernel evaluations for improved numerical and computational behavior.

Score estimation and score-based modeling. The score function $\nabla_x \log p(x)$ is a central object in statistics and machine learning. Score matching (Hyvärinen, 2005) provides a way to fit unnormalized models by matching scores rather than likelihoods, and denoising score matching connects score estimation to learning denoisers (Vincent, 2011). More recently, score-based generative models learn scores of noise-perturbed data distributions with neural networks and sample via Langevin or SDE-based dynamics (Song & Ermon, 2019; Song et al., 2021; Ho et al., 2020). These works typically target high-dimensional perceptual data and emphasize sampling or likelihood computation, whereas SD-KDE uses an explicit nonparametric density estimator and an analytically-defined score to improve pointwise density estimation. Related to our setting, Stein-based score estimators recover score information from samples via identities that avoid explicit density evaluation (Li & Turner, 2017). Our focus is complementary: we keep the statistical estimator fixed (SD-KDE and its Laplace variant) and address the practical bottleneck that arises when score-corrected estimators are deployed at large scale.

Fast evaluation of kernel sums and density models. The computational cost of KDE and related kernel methods is dominated by evaluating large Gaussian sums or kernel matrices. Algorithmic accelerations include multipole and series-expansion methods such as the Fast Gauss Transform (FGT) (Greengard & Strain, 1991) and improved variants tailored to Gaussian kernels (Yang et al., 2003). A different set of approaches uses randomized low-rank approximations (e.g., random Fourier features) to trade bias for computational efficiency (Rahimi & Recht, 2007). These methods reduce asymptotic complexity but can be sensitive to dimensionality, bandwidth, and target accuracy. In contrast, our work targets the exact SD-KDE computation and focuses on constant-factor reductions that make the estimator practical on modern accelerators.

GPU acceleration and tensor-core programming. A growing ecosystem of libraries and DSLs targets large kernel computations on GPUs. KeOps provides a memory-efficient reduction framework for kernel and distance matrices and is a strong baseline for Gaussian kernel reductions

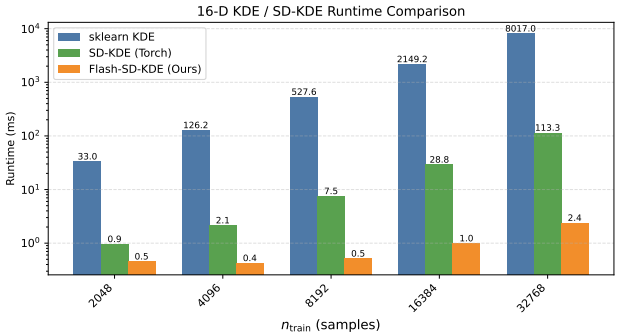

Figure 1. Runtime comparison for 16-D KDE/SD-KDE across n_{train} up to 32,768 ($n_{\text{test}} = n_{\text{train}}/8$).

at scale (Charlier et al., 2021). General deep learning frameworks such as PyTorch (Paszke et al., 2019) make it easy to express quadratic kernel computations, but their performance depends critically on whether the computation can be lowered to high-throughput primitives. Domain-specific languages such as Triton (Tillet et al., 2019) enable writing custom GPU kernels while still benefiting from compiler support for tiling and scheduling. At the hardware level, modern GPUs provide specialized matrix-multiply units (Tensor Cores) and mixed-precision execution paths that can offer large throughput improvements when computations are expressed in GEMM-like form (Micikevicius et al., 2017; Williams et al., 2009). Recent "flash"-style algorithms demonstrate that reordering computations to avoid materializing large intermediate matrices and to match the GPU memory hierarchy can yield substantial speedups for quadratic primitives (Dao et al., 2022). Our contribution follows this direction for score-corrected density estimation: we reorder SD-KDE to expose matrix-multiplication structure and implement a streaming accumulation strategy so that Tensor Cores can be used effectively without incurring prohibitive memory traffic.

3. Hardware

All experiments run on a workstation equipped with an NVIDIA RTX A6000 (GA102). This GPU exposes 84 streaming multiprocessors (SMs); each SM contains 128 FP32 ALUs and 16 special function units (SFUs). Because the ratio of FP32 ALUs to SFUs is 128:16, we treat one exp (issued on an SFU) as costing the equivalent of $128/16 = 8$ FP32 flops in our models. The card delivers roughly 40 TFLOP/s of peak FP32 throughput and approximately 770 GB/s of GDDR6 bandwidth.

The host CPU is a dual-socket AMD EPYC 7763 system ("Milan") configured with 2×64 cores and two hardware threads per core (256 logical CPUs reported by `lscpu`). The processors boost up to 3.53 GHz, expose 512 MiB of shared L3 cache across the sockets, and provide modern ISA extensions (AVX/AVX2, FMA, BMI1/2, SHA, VAES, etc.).

4. Method

We begin by recalling the standard KDE and SD-KDE definitions that the rest of the section will accelerate. Given samples $x_i \in \mathbb{R}^d$ and a bandwidth h, a Gaussian KDE is

$$\hat{p}(x) = \frac{1}{nh^d} \sum_{i=1}^{n} \varphi\left(\frac{x - x_i}{h}\right),$$

and SD-KDE forms debiased samples $x_i^{\text{SD}} = x_i + \frac{h^2}{2}\,\hat{s}(x_i)$ where $\hat{s}$ is an estimate of the score. In this work we always use an empirical score computed from the KDE itself. Assuming a standard Gaussian kernel φ and writing the KDE explicitly in terms of the samples, the empirical score is

$$\hat{s}(x) = \frac{\nabla_x \hat{p}(x)}{\hat{p}(x)} = \frac{\sum_{i=1}^{n} \left[-(x - x_i)\,\varphi\left(\frac{x-x_i}{h}\right)\right]}{h^2 \sum_{i=1}^{n} \varphi\left(\frac{x-x_i}{h}\right)}.$$

The score therefore has the same $O(n^2)$ pairwise structure as the KDE sum itself, which is the cost we target in what follows.

Our goal is to take n_{train} samples in d dimensions, form the empirical SD-KDE (score + shift), and evaluate the resulting density on n_{test} queries. Writing the computation in matrix form highlights the parallel structure. Let $x_i \in \mathbb{R}^d$ denote training points and $y_j \in \mathbb{R}^d$ denote query points, with bandwidth h and squared Euclidean distance

$$\|x_i - y_j\|^2 = \|x_i\|^2 + \|y_j\|^2 - 2\,x_i^\top y_j.$$

Stacking the training samples into $X \in \mathbb{R}^{n_{\text{train}} \times d}$, the empirical score computation is dominated by the train–train Gram matrix

$$G_{\text{score}} = XX^\top \in \mathbb{R}^{n_{\text{train}} \times n_{\text{train}}}.$$

Stacking queries into $Y \in \mathbb{R}^{n_{\text{test}} \times d}$ and denoting the debiased training samples by

$$X^{\text{SD}} = X + \tfrac{h^2}{2}\,\hat{s}(X),$$

the final SD-KDE evaluation uses the train–query Gram matrix

$$G_{\text{KDE}} = X^{\text{SD}} Y^\top \in \mathbb{R}^{n_{\text{train}} \times n_{\text{test}}}.$$

On modern NVIDIA GPUs this GEMM can be mapped to Tensor Cores (Micikevicius et al., 2017) and evaluated at 5–10$\times$ the throughput of standard FP32 SIMT arithmetic, particularly when we restrict to d that are multiples of 16 and use Triton's `tl.dot` interface (Tillet et al., 2019). The remaining operations—vector norms, broadcasted additions, and exponentials—are all $O(n_{\text{train}} n_{\text{test}})$ but quickly become a small fraction of the total FLOPs as d grows.

The numerator in the empirical score inherits a similar GEMM structure. Its numerator involves terms of the form

$$\sum_j -(x_i - x_j)\,\varphi_{ij}, \qquad \varphi_{ij} = \exp\!\left(-\frac{\|x_i - x_j\|^2}{2h^2}\right),$$

which naively suggests $O(n_{\text{train}} n_{\text{test}} d)$ additional elementwise arithmetic. Using the identity

$$\sum_j (x_i - x_j)\,\varphi_{ij} = x_i \sum_j \varphi_{ij} - \sum_j \varphi_{ij}\, x_j,$$

we can decompose the first term in the numerator into a elementwise sum, and the second term in the numerator into a second operation:

$$T = \Phi X.$$

Thus both the KDE evaluation and the SD-KDE score numerator reduce to Tensor-Core-accelerable matrix multiplies, plus $O(n^2)$ scalar work for the norms and exponentials. In this paper we focus on the $d = 16$ case, which aligns naturally with the Tensor Core tile sizes on the RTX A6000.

4.1. Arithmetic intensity in d dimensions

To understand when the high-dimensional SD-KDE becomes compute-bound, we estimate FLOPs, bytes moved, and arithmetic intensity for the d-dimensional case. Let $n_{\text{train}} = k$ and set $n_{\text{test}} = k/8$ as in the experiments.

Total FLOPs. The d-dimensional implementation consists of three main matrix-multiply stages:

1. Score Gram matrix $G_{\text{score}} = XX^\top$: $2dk^2$ FLOPs.
2. Score numerator $T = \Phi X$: $2dk^2$ FLOPs, plus $4k^2$ scalar FLOPs for norms and distance terms and $8k^2$ FLOPs for exponentials (counting each exp as 8 FLOPs due to the SFU/FP32 ratio).
3. Final KDE Gram matrix on debiased data: $2dk(k/8)$ FLOPs, plus $4k(k/8)$ scalar FLOPs and $8k(k/8)$ FLOPs for exponentials.

Aggregating these terms yields

$$\begin{aligned}\text{FLOPs}_d(k) &\approx 4dk^2 + 12k^2 + 2d\frac{k^2}{8} + 12\frac{k^2}{8} \\ &= \left(4d + 12 + \tfrac{d}{4} + \tfrac{3}{2}\right)k^2.\end{aligned}$$

Specifically for the $d = 16$ dimensional case, substituting $d = 16$ gives

$$\text{FLOPs}_{16}(k) \approx 81.5\,k^2,$$

which is on the order of 10^{11} FLOPs for $k = 32$k.

Bytes moved. To better capture implementation details, we count bytes per tile using the launch parameters that delivered the best runtime ($BLOCK\_M = 64$, $BLOCK\_N = 1024$). Each tile loads $64 \times d$ query values once ($4\,BLOCK\_Md$ bytes), streams $1024 \times d$ training values ($4\,BLOCK\_Nd$ bytes), and writes the partial PDF and weighted sums ($4\,BLOCK\_M$ and $4\,BLOCK\_Md$ bytes). All of this traffic goes to and from GDDR6, so for $d = 16$ a tile moves roughly

$$\begin{aligned}\text{Bytes}_{\text{tile}} &\approx 4\big(BLOCK\_Md + BLOCK\_Nd \\ &\qquad + BLOCK\_M + BLOCK\_Md\big) \\ &= 4\big(2\,BLOCK\_Md + BLOCK\_Nd \\ &\qquad + BLOCK\_M\big) \\ &\approx 7.4 \times 10^4 \text{ bytes.}\end{aligned}$$

The full problem processes $(k/BLOCK\_M) \times (k/BLOCK\_N)$ such tiles, so the total GDDR6 traffic is

$$\begin{aligned}\text{Bytes}_{16}(k) &\approx \text{Bytes}_{\text{tile}} \times \frac{k}{BLOCK\_M} \times \frac{k}{BLOCK\_N} \\ &\approx 1.13\,k^2 \text{ bytes.}\end{aligned}$$

Arithmetic intensity. Dividing FLOPs by bytes gives

$$\begin{aligned}I_d(k) &= \frac{\text{FLOPs}_d(k)}{\text{Bytes}_d(k)} \\ &\approx \frac{\left(4d + 12 + \frac{d}{4} + \frac{3}{2}\right)k^2}{4\left(\frac{9}{8}dk + \frac{k}{8}\right)} \\ &\sim C(d)\,k \quad \text{for large } k,\end{aligned}$$

with

$$C(d) \approx \frac{4d + 12 + d/4 + 3/2}{4 \cdot (9d/8)} = \frac{(17/4)\,d + 27/2}{9d/2}.$$

For $d = 16$ this simplifies to

$$I_{16}(k) \approx \frac{\text{FLOPs}_{16}(k)}{\text{Bytes}_{16}(k)} \approx \frac{81.5\,k^2}{1.13\,k^2} \approx 72 \text{ flops/byte.}$$

This tile-aware estimate more closely reflects the measured arithmetic intensity. Comparing against the A6000 specs (Tensor Core peak of 155 TFLOP/s versus $\approx$ 770 GB/s of memory bandwidth), the machine balance is roughly 200 flops/byte (Williams et al., 2009). Since our kernel sustains over 70 flops/byte, it lies well into the compute-bound regime on the RTX A6000. Nsight Compute reports an empirical arithmetic intensity of roughly 95 FLOPs/byte for the score kernel on the A6000, which is broadly in line with the 72 FLOPs/byte predicted by the tile-level model above. Minor differences stem from additional bookkeeping work (atomics, reductions) and from Nsight's finer-grained

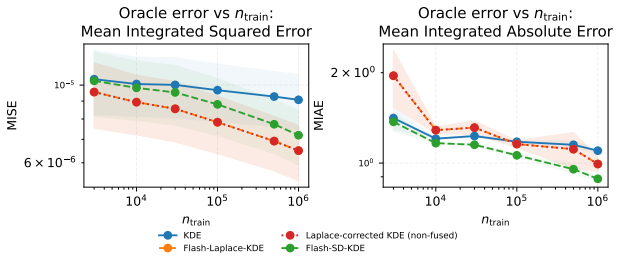

Figure 2. Oracle error on a 16D mixture-of-Gaussians benchmark. We report MISE and MIAE versus n_{train} for KDE, Flash-Laplace-KDE (fused Laplace correction), non-fused Laplace correction, and Flash-SD-KDE. The Laplace-corrected estimators can be slightly negative, so error is computed in a signed density manner.

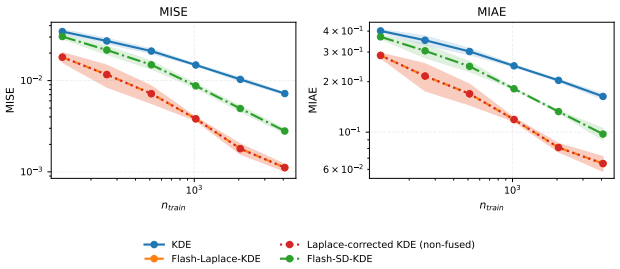

Figure 3. Oracle error on a 1D mixture-of-Gaussians benchmark. We report MISE and MIAE versus n_{train} for KDE, Flash-Laplace-KDE (fused Laplace correction), non-fused Laplace correction, and Flash-SD-KDE. The Laplace-corrected estimators can be slightly negative, so error is computed on the signed density.

accounting of texture/cache traffic, but both figures agree that the kernel operates far above the bandwidth roofline. Because the kernel mixes Tensor-Core GEMMs with FP32 scalar work (norms, exponentials, atomics), the effective machine balance sits between the tensor-core roof (≈ 200 flops/byte) and the FP32 roof (≈ 50 flops/byte). The observed 70–95 flops/byte therefore straddle these two limits: the GEMM portion is partially bandwidth-limited relative to Tensor Cores, while the scalar portion is compute-limited relative to FP32 ALUs.

5. Laplace-corrected KDE

Score-debiased KDE offers better asymptotic error rates, but its empirical score step adds a full quadratic pass over the data. A natural approximation is to replace the explicit debiasing with a correction to the kernel itself, in the spirit of classical higher-order bias corrections for KDE[1] (Fan & Hu, 1992; Jones & Signorini, 1997). This yields a Laplace-corrected KDE that captures the same leading-order bias reduction of SD-KDE (Epstein et al., 2025) while avoiding

[1]Laplace kernel has 0 second moment, but non-zero fourth moment, so it is a fourth-order kernel.

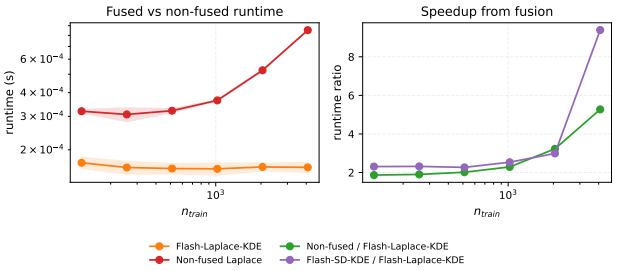

Figure 4. Runtime and speedup for Laplace correction in 1D. The left panel shows total runtime for the fused Flash-Laplace-KDE kernel and a non-fused implementation. The right panel reports speedup ratios, including Flash-SD-KDE relative to Flash-Laplace-KDE for context.

the empirical score computation at the cost of admitting possibly negative values.

Let $K_h(x)$ denote the isotropic Gaussian kernel in d dimensions.

$$K_h(x) = \frac{1}{(2\pi)^{d/2} h^d} \exp\left(-\frac{\|x\|^2}{2h^2}\right),$$

Let $\hat{p}(x) = \frac{1}{n}\sum_i K_h(x - x_i)$ be the corresponding KDE. The Laplace-corrected kernel is defined by

$$K_h^{\text{LC}}(x) = K_h(x) - \frac{h^2}{2}\Delta K_h(x).$$

For the Gaussian kernel, the Laplacian has a closed form.

$$K_h^{\text{LC}}(x) = K_h(x)\left(1 + \frac{d}{2} - \frac{\|x\|^2}{2h^2}\right),$$

The estimator becomes

$$\hat{p}_{\text{LC}}(x) = \frac{1}{n}\sum_{i=1}^{n} K_h^{\text{LC}}(x - x_i).$$

Connection to SD-KDE. SD-KDE (Epstein et al., 2025) forms debiased samples $x_i^{\text{SD}} = x_i + \frac{h^2}{2}\hat{s}(x_i)$ using an empirical score $\hat{s}$. Evaluating a KDE on the shifted samples and linearizing in h^2 yields a correction proportional to ΔK_h. When $\hat{s}$ is taken to be the KDE score, this linearized estimator coincides with the Laplace-corrected kernel above. Thus $\hat{p}_{\text{LC}}$ can be viewed as a first-order approximation to SD-KDE that preserves its leading bias reduction while avoiding an explicit score pass.

As an intuitive explanation of why both methods achieve the same leading bias reduction, we can first view vanilla KDE[2] as a heat semigroup (kernel) $P_t = e^{\frac{t}{2}\Delta}$ with $t = h^2$

[2]Throughout, we assume that the kernel is Gaussian for simplicity.

applying to the empirical density function f_n (sampled with replacement) to be

$$\hat{f} = P_t f_n.$$

From the linearity of the heat semigroup, we then have that the expected estimate

$$\mathbb{E}\left[\hat{f}\right] = \mathbb{E}\left[P_t f_n\right] = P_t \mathbb{E}\left[f_n\right] = P_t f,$$

where f is the true probability density function.

The bias is then, under a smoothness assumption we assume throughout,

$$\mathbb{E}\left[\hat{f}\right] - f = (P_t - I)f = \frac{t}{2}\Delta f + O(t^2).$$

The Laplace kernel deals with this leading term by using an operator $\left(I - \frac{t}{2}\Delta\right) P_t$ instead, making its bias

$$\mathbb{E}\left[\hat{f}^{\text{LC}}\right] - f = \left[\left(I - \frac{t}{2}\Delta\right) P_t - I\right] f = O(t^2).$$

The SD-KDE algorithm approaches this debias task differently by applying a **non-linear**[3] transportation kernel $T_{\frac{t}{2}\nabla \log f}$ before applying a standard heat kernel P_t, making its bias

$$\begin{aligned}
&\mathbb{E}\left[\hat{f}^{\text{SD-KDE}}\right] - f \\
&= \left[P_t T_{\frac{t}{2}\nabla f} - I\right] f \\
&= \left[P_t\left(f - \frac{t}{2}\nabla \cdot (f \nabla \log f) + O(t^2)\right) - f\right] \\
&= \left[P_t\left(f - \frac{t}{2}\Delta f + O(t^2)\right) - f\right] \\
&= \left[P_t\left(I - \frac{t}{2}\Delta\right) - I\right] f + O(t^2) \\
&= O(t^2).
\end{aligned}$$

Recall that $t = h^2$, so we have that Laplace KDE and SD-KDE have an $O(h^4)$ bias, while vanilla KDE has an $O(h^2)$ bias.

Note that the analysis above assumes that the SD-KDE has access to the oracle, so the translation kernel is linear. In practice, the score is empirical, making the translation kernel

$$T_{\frac{t}{2}\nabla \log(P_{t'} f_n)}$$

where $t' = \frac{h^2}{2}$ is the bandwidth for the kernel used for score estimation. Thus, the empirical SD-KDE is

$$\hat{f}^{\text{Emp SD-KDE}} = T_{\frac{t}{2}\nabla \log(P_{t'} f_n)} f_n,$$

which is no longer linear in f_n.

[3]Even when the transportation kernel has a non-linear transportation. The kernel itself is linear.

Why we consider this correction. The Laplace-corrected estimator is appealing when the full SD-KDE score is too expensive or when a fast bias-reduction surrogate is sufficient. It requires only the same pairwise distances and exponentials as standard KDE, plus a simple affine factor in the kernel value. The correction can be negative for large $\|x - x_i\|$, so $\hat{p}_{\text{LC}}$ is not guaranteed to be nonnegative, which we account for in our experiments.

Kernel fusion opportunity. Computing K_h^{LC} reuses the same scaled distances used to compute K_h. A fused kernel can compute $\phi = \exp(-\frac{1}{2}\,\text{scaled})$ and apply the Laplace factor inside the same pass. A non-fused implementation must either recompute distances or materialize intermediates in a second kernel. This increases memory traffic and kernel launches. We therefore treat the fused Laplace-corrected kernel as a distinct fast path and refer to it as *Flash-Laplace-KDE*.

6. Results

We now summarize the empirical behavior of the 16-D implementation on the RTX A6000. For each n_{train} in $\{2\text{k}, 4\text{k}, 8\text{k}, 16\text{k}, 32\text{k}\}$ we set $n_{\text{test}} = n_{\text{train}}/8$, draw data from a simple 16-D Gaussian mixture, and run three baselines: scikit-learn KDE (Pedregosa et al., 2011), Torch SD-KDE (GEMM-based, implemented in PyTorch (Paszke et al., 2019)), and Flash-SD-KDE (Tensor-Core GEMMs for both KDE and score, implemented in Triton (Tillet et al., 2019)). Figure 1 shows that the Flash-SD-KDE rapidly outpaces both baselines as n grows.

We also measure utilization by combining the flop model above with the measured runtimes. As shown in Figure 5, the Flash-SD-KDE GPU utilization is high into the multi-digit range once n_{train} exceeds 8k, despite a lot of operations such as exponential computations that can't utilize tensor cores. This confirms that the 16-D Tensor-Core formulation is firmly compute-bound and that additional tuning effort should focus on kernel fusion and occupancy rather than memory traffic.

To compare against a specialized KDE implementation, we also benchmarked PyKeOps LazyTensor operators at $n_{\text{train}} = 32\text{k}$, $n_{\text{test}} = 4\text{k}$. PyKeOps currently represents the state of the art for large-scale kernel reductions, so we report both its Gaussian KDE runtime and its SD-KDE runtime as a reference point. Table 1 shows that Flash-SD-KDE runs $1.57\times$ faster than the PyKeOps 16-D KDE baseline and $7.99\times$ faster than the PyKeOps 16-D SD-KDE implementation.

Method	Runtime (ms)	Rel. to Flash-SD-KDE
16-D Flash-SD-KDE	2.11	1×
PyKeOps 16-D KDE	3.33	1.4×
PyKeOps 16-D SD-KDE	16.91	7.99×

Table 1. Runtime comparison at $n_{\text{train}} = 32\text{k}$, $n_{\text{test}} = 4\text{k}$. PyKeOps rows report Gaussian KDE and SD-KDE (LazyTensor) runtimes, while Flash-SD-KDE executes the full SD-KDE pipeline.

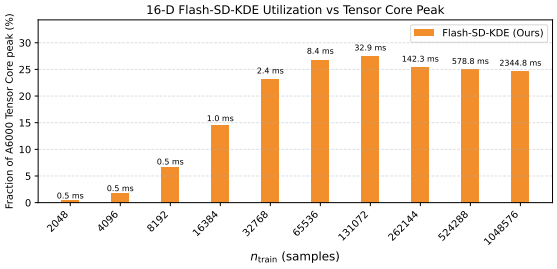

Figure 5. Utilization (percentage of RTX A6000 Tensor Core peak) for the 16-D SD-KDE pipeline, computed via the flop model from Section 4; bars are annotated with the observed runtimes (ms).

6.1. Oracle density benchmark for Laplace correction

Figure 2 reports oracle accuracy on the 16-D mixture benchmark as a function of n_{train}, using Mean Integrated Squared Error (MISE) on the left and Mean Integrated Absolute Error (MIAE) on the right. Across the sweep, Flash-SD-KDE substantially improves on vanilla KDE, indicating that the baseline KDE estimator is a poor fit in this regime. The Laplace-corrected variants achieve the lowest MISE throughout, and the fused Flash-Laplace-KDE curve overlaps the non-fused implementation, showing that fusion is an implementation optimization rather than a change in the estimator. In MIAE, Flash-SD-KDE attains the lowest error, while the Laplace-corrected estimators show a sharp drop between the smallest n_{train} and 10^4, followed by a small non-monotonic "kink" (a slight increase at intermediate n_{train}). The kink lies within the uncertainty bands and is consistent with finite-sample variability and the greater sensitivity of an absolute-error metric to the signed tail correction. Both metrics continue to decrease as n_{train} grows. Because the Laplace-corrected kernel is not guaranteed to remain non-negative, we compute MISE/MIAE on the signed estimator and separately log the integrated negative mass as a diagnostic.

We also evaluate Laplace-corrected KDE on a 1D oracle density. Figure 3 compares KDE, Flash-Laplace-KDE (fused Laplace correction), non-fused Laplace correction, and Flash-SD-KDE. The Laplace-corrected estimators can take small negative values for large $\|x - x_i\|$. We therefore compute errors on the signed density, and we report the negative-mass diagnostics separately in the benchmark logs.

Figure 4 shows the runtime advantage of fusion. The fused Flash-Laplace-KDE kernel consistently outperforms the non-fused Laplace correction across n_{train}. For context, the same plot also reports the Flash-SD-KDE to Flash-Laplace-KDE runtime ratio.

6.2. Performance optimizations

Nsight Systems traces show that roughly 95% of the SD-KDE runtime at $n_{\text{train}} = 32\text{k}$, $n_{\text{test}} = 4\text{k}$ is spent computing the empirical score. We therefore concentrated optimization effort on the score kernel, sweeping the launch parameters to raise utilization. Specifically we varied:

- $BLOCK\_M \in \{32, 64, 128, 256\}$,
- $BLOCK\_N \in \{32, 64, 128, 256, 512, 1024\}$,
- $num\_warps \in \{1, 2, 4, 8\}$,
- $num\_stages \in \{1, 2, 4\}$,

and selected the combination that minimized runtime. For this workload we found that $BLOCK\_M = 64$, $BLOCK\_N = 1024$, $num\_warps = 2$, and $num\_stages = 2$ gave the best overall performance, increasing measured FLOP utilization by more than $2\times$ relative to smaller-tile settings.

Beyond the launch-parameter sweep, two design choices account for most of the speedup over a standard tiling implementation:

1. **Tensor-Core tiling:** all high-dimensional dot products are processed as 16×16 tiles with Tensor Core acceleration, allowing the GEMM portion of the score and KDE kernels to run near the TF32 roofline.

2. **Streaming accumulation:** we never materialize full $n_{\text{train}} \times n_{\text{train}}$ (or query) matrices; instead we stream tiles through registers and rely on atomic reductions to keep global-memory traffic linear in n.

Together these optimizations push the kernels close to the hardware limit. Nsight Compute's "Speed of Light" report shows $\sim 68\%$ SM throughput and roughly 90% L1 throughput for the score kernel at $n_{\text{train}} = 32\text{k}$ and $n_{\text{test}} = 4\text{k}$, which is consistent with the tensor-core utilization trends in Figure 5: we cannot reach the nominal Tensor Core peak because the kernel still executes substantial non-Tensor-Core work (norms, exponentials, atomics). At these utilization levels we stopped further low-level tuning, since additional optimizations would likely yield only modest incremental gains.

7. Conclusion

Score-debiased kernel density estimation (SD-KDE) offers improved asymptotic accuracy over classical KDE, but its reliance on an empirical score has historically made it substantially slower in practice. This paper shows that the dominant cost of empirical SD-KDE is not inherently prohibitive: by re-ordering the computation to expose matrix-multiplication structure, both KDE evaluation and the score numerator can be expressed as Tensor Core–accelerable GEMMs plus lightweight elementwise operations, while avoiding materializing full pairwise interaction matrices via streaming accumulation. In the 16-D setting, this hardware-aligned formulation yields large end-to-end gains—up to $47\times$ faster than a strong Torch SD-KDE baseline and $3{,}300\times$ faster than scikit-learn KDE—and scales to previously infeasible regimes, completing SD-KDE on $\sim$1M training points and $\sim$131k queries in 2.3 s on a single GPU.

Overall, our results suggest that practical nonparametric estimators can often be unlocked by hardware-aware reformulations that maximize GPU utilization. Future directions include extending the approach beyond d multiples of 16, supporting richer bandwidth parameterizations and kernels, further reducing non-Tensor-Core overheads (e.g., exponentials and atomics), and developing nonnegativity-preserving approximations and multi-GPU variants.

8. Acknowledgements

The authors would like to thank Thanawat Sornwanee for helpful discussions.

Impact Statement

This paper presents work whose goal is to accelerate SD-KDE and KDE kernels on GPUs. We expect the primary impact to be enabling practitioners to apply these estimators to larger datasets; we do not foresee unique societal consequences outside the usual considerations for density estimation applications.

References

Abramson, I. S. On bandwidth variation in kernel estimates—a square root law. *The Annals of Statistics*, 10(4):1217–1223, 1982. doi: 10.1214/aos/1176345986.

Backurs, A., Indyk, P., and Wagner, T. Space and time efficient kernel density estimation in high dimensions. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

Botev, Z. I., Grotowski, J. F., and Kroese, D. P. Kernel density estimation via diffusion. *The Annals of Statistics*, 38(5):2916–2957, 2010. doi: 10.1214/10-AOS799.

Charikar, M. and Siminelakis, P. Hashing-based-estimators for kernel density in high dimensions. In *2017 IEEE 58th Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 1032–1043, 2017. doi: 10.1109/FOCS.2017.99.

Charikar, M. and Siminelakis, P. Multi-resolution hashing for fast pairwise summations. In *2019 IEEE 60th Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 769–792, 2019. doi: 10.1109/FOCS.2019.00051.

Charikar, M., Kapralov, M., Nouri, N., and Siminelakis, P. Kernel density estimation through density constrained near neighbor search. In *2020 IEEE 61st Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 172–183, 2020. doi: 10.1109/FOCS46700.2020.00025.

Charlier, B., Feydy, J., Glaunès, J. A., Collin, F.-D., and Durif, G. Kernel operations on the gpu, with autodiff, without memory overflows. *Journal of Machine Learning Research*, 22(74):1–6, 2021. URL http://jmlr.org/papers/v22/20-275.html.

Coleman, B. and Shrivastava, A. Sub-linear RACE sketches for approximate kernel density estimation on streaming data. In *Proceedings of The Web Conference 2020 (WWW)*, pp. 1739–1749, 2020. doi: 10.1145/3366423.3380244.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. Flashattention: Fast and memory-efficient exact attention with IO-awareness. *arXiv preprint arXiv:2205.14135*, 2022.

Dwivedi, R. and Mackey, L. Kernel thinning. In *Proceedings of the 34th Conference on Learning Theory (COLT)*, volume 134, 2021.

Epstein, E. L., Dwaraknath, R. V., Sornwanee, T., Winnicki, J., and Liu, J. W. SD-KDE: Score-debiased kernel density estimation. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2025. URL https://openreview.net/forum?id=fvho95EtPu.

Fan, J. and Hu, T.-C. Bias correction and higher order kernel functions. *Statistics & Probability Letters*, 13(3):235–243, 1992. doi: 10.1016/0167-7152(92)90053-8.

Greengard, L. and Strain, J. The fast gauss transform. *SIAM Journal on Scientific and Statistical Computing*, 12(1):79–94, 1991. doi: 10.1137/0912004.

Ho, J., Jain, A., and Abbeel, P. Denoising diffusion probabilistic models. *arXiv preprint arXiv:2006.11239*, 2020.

Hyvärinen, A. Estimation of non-normalized statistical models by score matching. *Journal of Machine Learning Research*, 6:695–709, 2005.

Jones, M. C. and Signorini, D. F. A comparison of higher-order bias kernel density estimators. *Journal of the American Statistical Association*, 92(439):1063–1073, 1997. doi: 10.1080/01621459.1997.10474062.

Kacham, P., Mirrokni, V., and Zhong, P. PolySketchFormer: Fast transformers via sketching polynomial kernels. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*, 2024.

Li, Y. and Turner, R. E. Gradient estimators for implicit models. *arXiv preprint arXiv:1705.07107*, 2017.

Luo, C. and Shrivastava, A. Arrays of (locality-sensitive) count estimators (ACE): Anomaly detection on the edge. In *Proceedings of the 2018 World Wide Web Conference (WWW)*, pp. 1439–1448, 2018. doi: 10.1145/3178876.3186056.

Micikevicius, P., Narang, S., Alben, J., Diamos, G., Elsen, E., Garcia, D., Ginsburg, B., Houston, M., Kuchaiev, O., Venkatesh, G., and Wu, H. Mixed precision training. *arXiv preprint arXiv:1710.03740*, 2017.

Parzen, E. On estimation of a probability density function and mode. *The Annals of Mathematical Statistics*, 33(3):1065–1076, 1962. doi: 10.1214/aoms/1177704472.

Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L., Desmaison, A., Köpf, A., Yang, E., DeVito, Z., Raison, M., Tejani, A., Chilamkurthy, S., Steiner, B., Fang, L., Bai, J., and Chintala, S. Pytorch: An imperative style, high-performance deep learning library. *arXiv preprint arXiv:1912.01703*, 2019.

Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Müller, A., Nothman, J., Louppe, G., Prettenhofer, P., Weiss, R., Dubourg, V., VanderPlas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., and Duchesnay, É. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

Rahimi, A. and Recht, B. Random features for large-scale kernel machines. In *Advances in Neural Information Processing Systems*, 2007.

Rosenblatt, M. Remarks on some nonparametric estimates of a density function. *The Annals of Mathematical Statistics*, 27(3):832–837, 1956. doi: 10.1214/aoms/1177728190.

Shetty, A., Dwivedi, R., and Mackey, L. Distribution compression in near-linear time. In *Proceedings of the 10th International Conference on Learning Representations (ICLR)*, 2022.

Silverman, B. W. *Density Estimation for Statistics and Data Analysis*. Chapman and Hall, 1986. doi: 10.1201/9781315140919.

Siminelakis, P., Rong, K., Bailis, P., Charikar, M., and Levis, P. Rehashing kernel evaluation in high dimensions. In *Proceedings of the 36th International Conference on Machine Learning (ICML)*, volume 97, pp. 5789–5798, 2019.

Song, Y. and Ermon, S. Generative modeling by estimating gradients of the data distribution. *arXiv preprint arXiv:1907.05600*, 2019.

Song, Y., Sohl-Dickstein, J., Kingma, D. P., Kumar, A., Ermon, S., and Poole, B. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2021. ICLR 2021.

Tillet, P., Kung, H. T., and Cox, D. Triton: An intermediate language and compiler for tiled neural network computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages*, 2019. doi: 10.1145/3315508.3329973.

Vincent, P. A connection between score matching and denoising autoencoders. *Neural Computation*, 23(7):1661–1674, 2011. doi: 10.1162/NECO_a_00142.

Wand, M. P. and Jones, M. C. *Kernel Smoothing*. Chapman and Hall, 1995. doi: 10.1201/b14876.

Williams, S., Waterman, A., and Patterson, D. Roofline: An insightful visual performance model for multicore architectures. *Communications of the ACM*, 52(4):65–76, 2009. doi: 10.1145/1498765.1498785.

Yang, C., Duraiswami, R., Gumerov, N. A., and Davis, L. Improved fast gauss transform and efficient kernel density estimation. In *Proceedings of the Ninth IEEE International Conference on Computer Vision*, pp. 664–671, 2003. doi: 10.1109/ICCV.2003.1238383.

Zandieh, A., Han, I., Daliri, M., and Karbasi, A. KDEformer: Accelerating transformers via kernel density estimation. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202, pp. 40605–40623, 2023.

A. Flash-SD-KDE in low dimensions

For completeness we briefly summarize the 1-D SD-KDE arithmetic intensity and empirical behavior. With $n_{\text{train}} = k$ and $n_{\text{test}} = k/8$, there are two steps:

1. **Score + shift:** For each training point we compute $\hat{s}(x_i)$ from all k points and then form $x_i^{\text{SD}} = x_i +$

$\frac{h^2}{2}\hat{s}(x_i)$. This uses $O(k^2)$ pairwise kernel interactions. With the RTX A6000 hardware ratio (128 FP32 ALUs, 16 SFUs per SM) we budget an exp as 8 flop-equivalents. The score accumulation requires one exp and roughly eight additional arithmetic ops (subtraction, scaling, accumulation), yielding $c_1 \approx 16$ flops per (train, train) pair.

2. **KDE on debiased samples:** We then evaluate a standard Gaussian KDE at $k/8$ query points using the k debiased samples, which uses $O(k^2/8)$ interactions. Each pair requires one exp (8 flops) plus about six other operations (difference, square, scaling, accumulation), so we take $c_2 \approx 14$ flops per (train, test) pair.

The total work is therefore approximated by

$$\text{FLOPs}(k) \;\approx\; c_1 k^2 + c_2\, k\,(k/8) \;\approx\; 16k^2 + 14\frac{k^2}{8} \;=\; 17.75\,k^2.$$

For $k = 32$k this is on the order of 2×10^{10} flops. When we fix $n_{\text{test}} = k/8$ we move approximately $5k$ bytes (counting one read of each train and test point and one write of each output), so the arithmetic intensity scales as

$$I(k) = \frac{\text{FLOPs}(k)}{\text{Bytes}(k)} \;\approx\; \frac{17.75\,k^2}{5\,k} \approx 3.55\,k \quad \text{flops/byte},$$

placing realistic problem sizes firmly in the compute-bound regime.

Empirically we sweep k over powers of two from 1024 to 64k, with $n_{\text{test}} = k/8$, and average over three seeds. For each configuration we record scikit-learn Gaussian KDE time, and SD-KDE Torch time, and Flash-SD-KDE time. Across the entire range, the Flash-SD-KDE implementation is consistently faster than scikit-learn, with speedups growing with k; at the largest sizes, Flash-SD-KDE achieves around 2 orders-of-magnitude speedup relative to the sklearn baseline, as shown in Figure 6.

As shown in Figure 7, we also compare the utilization of Flash-SD-KDE with SD-KDE (Torch) over a range of 1-D problem sizes, using powers of two for n_{train} up to $2^{16} \approx 64$ thousand (and $n_{\text{test}} = n_{\text{train}}/8$).

B. Additional Experiments

This appendix reports additional measurements that complement the benchmarks in Section 6. We extend the runtime sweep to stronger exact baselines and to a second GPU, isolate the contributions of streaming and Tensor-Core tiling through controlled ablations, and evaluate Flash-SD-KDE on a real-data anomaly detection benchmark. All measurements use the 16-D pipeline. Unless stated otherwise, runtimes are reported in milliseconds, exclude one-time warmup and JIT compilation costs, and use the 16-D Gaussian-mixture benchmark of Section 6 with $n_{\text{test}} = n_{\text{train}}/8$.

B.1. Extended baseline sweep on the RTX A6000

To place Flash-SD-KDE against a fuller set of exact baselines, we extend the PyKeOps comparison from a single point to the full n_{train} sweep and add a `torch.compile` variant of the GEMM-based PyTorch baseline. Table 2 reports runtimes on the RTX A6000.

Flash-SD-KDE is the fastest exact method at every problem size in the sweep. At $n_{\text{train}} = 32{,}768$, it is roughly $7.7\times$ faster than PyKeOps, $12.2\times$ faster than `torch.compile`, $40.6\times$ faster than the eager PyTorch baseline, and over three orders of magnitude faster than scikit-learn. The PyTorch-based baselines slow down rapidly with n_{train}, while PyKeOps and Flash-SD-KDE both scale gracefully; among these two, Flash-SD-KDE retains a consistent margin throughout the sweep.

B.2. Generalization across GPU families

Section 6 measures performance on the RTX A6000. To check that the gains are not specific to that architecture, we rerun the same sweep on a Tesla V100-PCIE-16GB. The V100 has a different SFU/FP32 ratio and a smaller Tensor-Core throughput than the A6000, so it provides an independent stress test of the implementation. Table 3 reports the result.

The qualitative ordering is preserved: Flash-SD-KDE remains the fastest method at every measured size on the V100, with margins comparable to those seen on the A6000. The eager PyTorch SD-KDE baseline is competitive on the V100 at small n_{train} but degrades rapidly with n_{train}, while `torch.compile` pays a large warm overhead that the exclusion of JIT costs alone does not remove.

B.3. Effect of query batch size

Many KDE workloads operate at small or moderate query batch sizes, in which case the score pass dominates and the cost of batching across queries is a secondary effect. We isolate this by fixing $n_{\text{train}} = 32{,}768$ and sweeping n_{test} across four orders of magnitude. Table 4 reports the result.

Flash-SD-KDE remains in the 1.91–2.75 ms range across a $4{,}096\times$ sweep of n_{test}, while every baseline shows a noticeable rise at the largest query batch. This pattern is consistent with the kernel-level analysis in Section 4: the score pass scales as $O(n_{\text{train}}^2)$ and is independent of n_{test}, so when the score dominates (*i.e.* at large n_{train} relative to n_{test}), the end-to-end runtime is essentially flat in n_{test}. Flash-SD-KDE is therefore particularly attractive in workflows that score a small number of queries against a large training set, such as

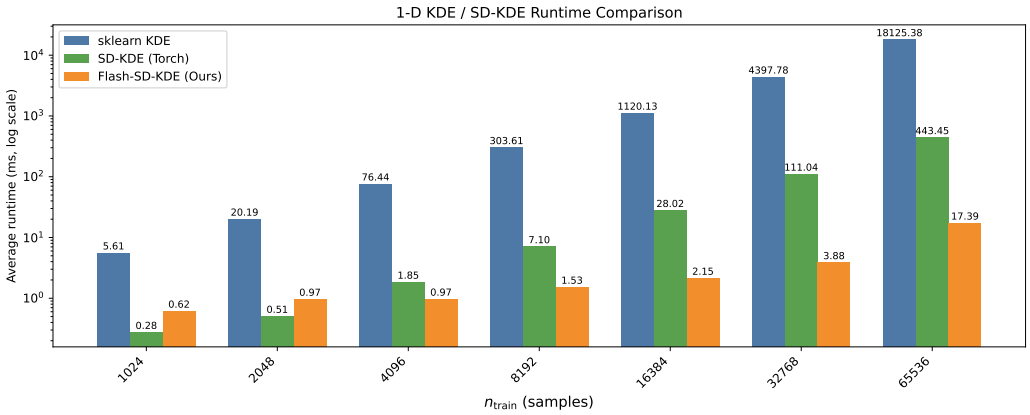

Figure 6. Average runtime (log-scale y-axis) of scikit-learn KDE, SD-KDE (Torch), and Flash-SD-KDE across n_{train} in 1-D; annotations show observed runtimes.

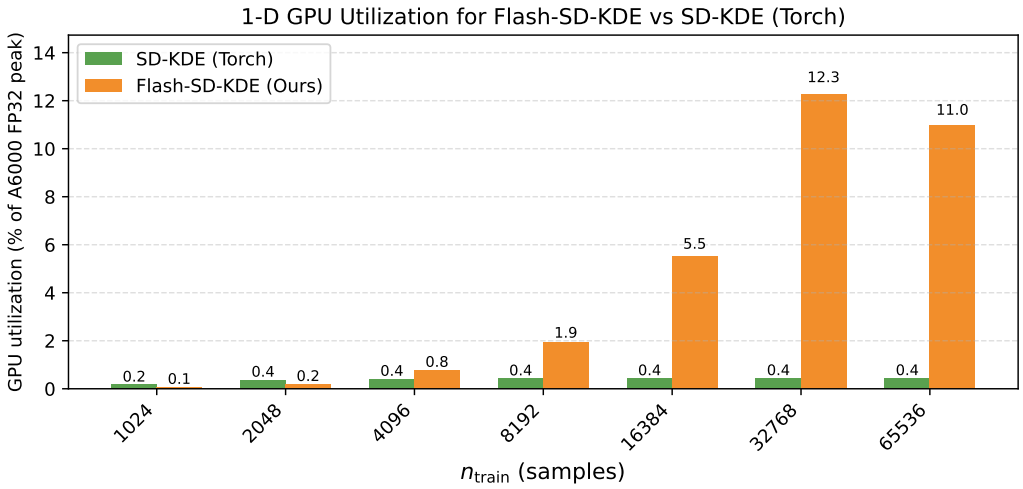

Figure 7. Utilization of the 1-D Flash-SD-KDE kernel and SD-KDE (Torch) for large n_{train} (powers of two up to 2^{16}). Labels show the observed runtime for each data size; error bars are omitted because a single seed is used per point.

anomaly or out-of-distribution scoring.

B.4. Streaming versus full materialization

The streaming design described in Section 4 avoids ever materializing the full pairwise kernel matrices. To quantify how much of the speedup is attributable to streaming alone, we compare the streamed Flash-SD-KDE kernel against an explicit "full-materialization" baseline that allocates the $n_{\text{train}} \times n_{\text{train}}$ training kernel matrix and the $n_{\text{test}} \times n_{\text{train}}$ query kernel matrix. Both implementations otherwise share the same numerical formulation. Table 5 reports runtime and peak GPU memory above the resident input-tensor footprint at the largest size we can materialize on a single A6000, $n_{\text{train}} = 65{,}536$, $n_{\text{test}} = 8{,}192$.

Streaming reduces peak extra GPU memory by roughly $1{,}000\times$ at this size, from $\sim$16 GB to $\sim$16 MB, and gives a $55.7\times$ runtime speedup over the materialized Tensor-Core variant. The materialized baseline cannot scale meaningfully beyond this size on the A6000, while streaming Flash-SD-KDE continues to operate well under typical GPU memory budgets. Streaming therefore accounts for the bulk of the memory savings reported in Section 6, separately from the Tensor-Core speedup studied in Section B.5.

B.5. Tensor-Core ablation within the streamed kernel

Within the streamed Flash-SD-KDE kernel we additionally compare a Tensor-Core GEMM path against an otherwise identical FP32 path that disables Tensor-Core tiling. This isolates the Tensor-Core contribution from streaming and from the launch-parameter tuning of Section 4. Table 6 sweeps n_{train} from 2,048 to 65,536 with $n_{\text{test}} = n_{\text{train}}/8$.

The Tensor-Core advantage grows from a marginal $1.04\times$ at

n_{train}	n_{test}	sklearn	Torch	`torch.compile`	PyKeOps	Flash-SD-KDE
2,048	256	30.7	1.1	6.0	1.9	0.7
4,096	512	132.6	2.2	2.9	2.5	0.5
8,192	1,024	509.7	7.5	5.0	3.6	0.5
16,384	2,048	1,895.7	29.0	11.3	6.7	1.1
32,768	4,096	7,292.7	113.6	34.3	21.7	2.8

Table 2. Average per-call runtime (ms) of exact 16-D SD-KDE implementations on the RTX A6000, lower is better. Columns are scikit-learn Gaussian KDE, the GEMM-based PyTorch SD-KDE, the same PyTorch implementation under `torch.compile`, the PyKeOps LazyTensor SD-KDE, and our Flash-SD-KDE. Data are drawn from the 16-D Gaussian-mixture benchmark of Section 6, with $n_{\text{test}} = n_{\text{train}}/8$. One-time warmup and JIT compilation costs are excluded.

n_{train}	n_{test}	sklearn	Torch	`torch.compile`	PyKeOps	Flash-SD-KDE
2,048	256	51.60	0.55	65.20	3.29	0.51
4,096	512	270.94	1.78	63.74	4.15	0.62
8,192	1,024	810.39	6.51	54.25	5.51	1.19
16,384	2,048	3,362.95	25.31	59.88	11.19	3.92

Table 3. Average per-call runtime (ms) on a Tesla V100-PCIE-16GB, using the same 16-D Gaussian-mixture benchmark and $n_{\text{test}} = n_{\text{train}}/8$ protocol as Table 2. Lower is better. One-time warmup and JIT costs are excluded.

the smallest size to $4.94\times$ at $n_{\text{train}} = 32{,}768$, and stabilizes near $5\times$ at the largest sizes we can run. This is consistent with arithmetic-intensity reasoning: at small n_{train} the GEMM tiles are too small to saturate Tensor Cores, while at moderate-to-large n_{train} the GEMM portion dominates and the kernel approaches the mixed Tensor-Core/FP32 roofline analysed in Section 4. Together with Section B.4, this gives a clean attribution: streaming is responsible for the memory-footprint reduction at all sizes, while Tensor-Core tiling drives the runtime gain at scale.

We do not currently fuse the score pass with the KDE pass into a single kernel. Nsight Systems traces show that the score pass accounts for roughly 90% of the end-to-end runtime, so fusing the two kernels yields only a small incremental gain at the cost of substantial additional code complexity. We retain the simpler two-kernel structure.

B.6. Anomaly detection on the ACE benchmark

The benchmarks above measure runtime and oracle accuracy. To check that Flash-SD-KDE is competitive on a real, non-synthetic task, we evaluate it on the anomaly-detection benchmark of Luo & Shrivastava (2018). We follow the same protocol: rank all points by an outlier score, threshold at the top-K to recover a candidate outlier set, and report the number of true outliers correctly recovered, the total number reported, the number missed, and the wall-clock execution time. Flash-SD-KDE produces an outlier score by negating its density estimate, so that low-density points rank as outliers.

We compare against the twelve baselines reported in the ACE paper on two of the three datasets evaluated there: the Statlog Shuttle dataset ($n = 34{,}987$, $d = 9$, 879 outliers) and the ALOI object-image dataset ($n = 50{,}000$, $d = 27$, $1{,}508$ outliers). The non-Flash-SD-KDE rows in Tables 7–8 are taken directly from the ACE paper; the Flash-SD-KDE rows are our measurements on the RTX A6000 using the same datasets and the same top-K protocol.

On Shuttle, Flash-SD-KDE recovers 807 of the 879 true outliers at the chosen threshold, compared with 610 for the strongest non-Flash baseline (kNNW) and 273 for ACE itself, while running in 0.60 s versus the next fastest method, ACE at 0.81 s. On ALOI, Flash-SD-KDE is competitive on recovery quality (within 20% of the best non-Flash baseline) and is more than an order of magnitude faster than ACE, and roughly three orders of magnitude faster than the distance/density-based baselines. The two datasets together cover relatively low ($d = 9$) and moderate ($d = 27$) dimensionalities, both of which are within the regime where the Tensor-Core formulation of Section 4 is effective.

We additionally evaluate Flash-SD-KDE at large scale on a public point-anomaly release of the KDD-Cup 99 HTTP dataset ($n = 620{,}098$, $d = 29$, $1{,}052$ labelled anomalies), which is comparable in scale to the $596{,}853$-point KDD HTTP subset used in the ACE paper. On this release Flash-SD-KDE reaches a ROC-AUC of 0.92 in 7.27 s of end-to-end execution time on the RTX A6000. For reference, the ACE paper reports 23.33 s on its own KDD HTTP subset. We do not claim a like-for-like comparison here because the public release does not match the exact subset used in the ACE paper, but the result indicates that Flash-SD-KDE remains practical at the $\sim 10^6$-point scale on commodity hardware.

n_{train}	n_{test}	Torch	`torch.compile`	PyKeOps	Flash-SD-KDE
32,768	4	100.97	32.44	15.64	1.92
32,768	16	101.00	32.41	15.43	1.91
32,768	64	103.27	33.44	15.48	1.91
32,768	256	104.09	33.68	16.11	1.92
32,768	1,024	106.22	33.69	15.81	1.97
32,768	4,096	116.81	35.97	15.85	2.11
32,768	16,384	158.18	43.14	16.79	2.75

Table 4. Average per-call runtime (ms) on the RTX A6000 as a function of the query batch size n_{test}, with $n_{\text{train}} = 32{,}768$ held fixed. Lower is better. Data are drawn from the 16-D Gaussian-mixture benchmark.

Method	Runtime (ms)	Peak alloc (MB)
Streamed Flash + Tensor Cores	10.99	16.25
Streamed Flash + No Tensor Cores	49.52	16.25
Full materialization + Tensor Cores	611.63	16,400.50
Full materialization + No Tensor Cores	617.49	16,400.50

Table 5. Runtime and peak extra GPU memory at $n_{\text{train}} = 65{,}536$, $n_{\text{test}} = 8{,}192$ on the RTX A6000. "Streamed Flash" is the production Flash-SD-KDE kernel; "Full materialization" explicitly forms the $n_{\text{train}} \times n_{\text{train}}$ and $n_{\text{test}} \times n_{\text{train}}$ kernel matrices in global memory. Peak alloc is reported above the input-tensor footprint and excludes shared workspace.

n_{train}	n_{test}	TC (ms)	no-TC (ms)	no-TC / TC
2,048	256	0.30	0.31	1.04×
4,096	512	0.28	0.35	1.24×
8,192	1,024	0.29	0.87	2.99×
16,384	2,048	0.66	2.83	4.27×
32,768	4,096	2.12	10.48	4.94×
65,536	8,192	9.30	44.59	4.80×

Table 6. Tensor-Core ablation within the streamed Flash-SD-KDE kernel on the RTX A6000. "TC" is the production kernel; "no-TC" is an otherwise identical kernel with Tensor-Core tiling disabled. The right-most column reports $\text{runtime}_{\text{no-TC}}/\text{runtime}_{\text{TC}}$, so values above $1\times$ mean the Tensor-Core path is faster. Lower runtime is better.

C. Additional Literature Review

This appendix expands the related-work discussion of Section 2 along three axes that are most relevant to score-debiased and other bias-corrected density estimators: sub-quadratic approximate KDE algorithms, sample compression and coreset methods, and KDE-as-primitive applications in modern machine learning. We also discuss potential hybrids of these algorithmic directions with the hardware-accelerated exact pipeline of Section 4. The discussion is intentionally complementary to the experimental focus of the main paper: the works below approximate either the kernel sum or the training set, while our work makes the exact SD-KDE estimator practical at scale.

C.1. Sub-quadratic approximate KDE via locality-sensitive hashing

A line of work starting with the hashing-based estimator (HBE) framework of Charikar & Siminelakis (2017) reduces the cost of evaluating a Gaussian kernel sum from quadratic to sub-quadratic by sampling from locality-sensitive hash buckets. At a high level, HBE hashes both the training set and the query point into LSH buckets whose collision probability is calibrated to the kernel, and uses bucket cardinalities together with a small number of refinement samples to estimate the kernel sum to a target relative error. Backurs et al. (2019) improve the construction so that it requires only linear preprocessing time and space while retaining the same query time, and Siminelakis et al. (2019) bring the scheme closer to practice through adaptive sample-size selection and sharper data-dependent variance bounds. Charikar & Siminelakis (2019) extend the construction beyond Gaussian-like kernels to a much broader class of pairwise summations using a multi-resolution hashing scheme. A subsequent refinement (Charikar et al., 2020) casts the sub-quadratic KDE problem as a density-constrained near-neighbor search and removes the redundancy that arises in HBE when nearby points fall into different buckets on opposite sides of the query.

The ACE method of Luo & Shrivastava (2018), which we evaluate against in Section B.6, is a practical instantiation of this family specialized for streaming anomaly detection: rather than estimating the kernel sum, it tracks per-bucket counts and uses them as proxies for local density. The closely related RACE sketch of Coleman & Shrivastava (2020) extends the LSH-counting view to streaming KDE with explicit error guarantees. ACE was a natural choice for our anomaly-detection comparison precisely because it shares the high-level inverse-density-as-anomaly viewpoint of a KDE-based scorer while bringing the sub-quadratic LSH cost.

Method	Reported	Correct	Missed	Time (s)	vs ACE
ACE	6,763	273	606	0.81	1×
LOF	4,356	381	498	14.12	17.4×
kNN	4,897	493	386	12.35	15.2×
kNNW	5,264	610	269	13.54	16.7×
LoOP	6,145	201	678	14.51	17.9×
LDOF	6,433	330	549	16.42	20.3×
ODIN	9,775	375	504	12.21	15.1×
KDEOS	12,630	314	565	11.73	14.5×
COF	9,133	280	599	13.45	16.6×
LDF	9,809	375	504	19.93	24.6×
INFLO	4,488	183	696	14.03	17.3×
FastVOA	8,532	271	608	235.10	290.2×
Flash-SD-KDE	5,130	807	72	0.60	0.74×

Table 7. Anomaly recovery on Statlog Shuttle ($n = 34{,}987$, $d = 9$, 879 true outliers). "Reported" is the number of points returned at the chosen threshold, "Correct" the number of those that are true outliers, and "Missed" the number of true outliers not recovered. Time is total execution time in seconds; "vs ACE" is the runtime ratio relative to the ACE row. Non-Flash-SD-KDE rows are reproduced from the ACE paper; Flash-SD-KDE is measured on the RTX A6000.

Method	Reported	Correct	Missed	Time (s)	vs ACE
ACE	7,216	340	1,168	1.26	1×
LOF	4,476	519	989	72.31	57.4×
kNN	5,428	447	1,061	63.27	50.2×
kNNW	5,558	329	1,508	89.96	71.4×
LoOP	5,121	253	1,179	59.97	47.6×
LDOF	7,501	470	1,038	60.39	47.9×
ODIN	10,110	162	1,346	72.69	57.6×
KDEOS	9,515	404	1,104	55.89	44.4×
COF	8,746	284	1,224	81.74	64.9×
LDF	9,133	301	1,207	60.51	48.0×
INFLO	10,328	420	1,088	72.13	57.2×
FastVOA	8,931	319	1,189	291.10	231.0×
Flash-SD-KDE	7,250	437	1,071	0.09	0.07×

Table 8. Anomaly recovery on the ALOI object-image dataset ($n = 50{,}000$, $d = 27$, 1,508 true outliers). Columns and source attribution match Table 7.

These methods are complementary to ours rather than substitutes. HBE-family algorithms approximate *classical* KDE, so applying them out of the box to SD-KDE would forfeit the bias-reduction guarantees that motivate SD-KDE in the first place. A natural hybrid direction (Section C.5) is to use a sub-quadratic approximation only for the empirical-score pass and retain exact streaming Tensor-Core evaluation for the final density.

C.2. Sample compression, coresets, and kernel thinning

A second line of work reduces the cost of kernel computations by replacing the full training set with a much smaller weighted subset. Dwivedi & Mackey (2021) introduce kernel thinning, which produces a coreset of size $O(\sqrt{n})$ whose worst-case kernel-mean error matches that of the full sample up to logarithmic factors, by halving the dataset under a self-balancing process and selecting the half that better preserves a target reproducing-kernel Hilbert space functional. Shetty et al. (2022) extend this construction with a meta-procedure (Compress++) that reduces the total runtime to near-linear while inflating the integration error by at most a constant factor, making kernel thinning practical at substantially larger scales.

Compression-based methods affect a different cost axis from the one we target. They reduce n before any kernel evaluations are performed, while we keep n fixed and accelerate the resulting $O(n^2)$ computation. For SD-KDE specifically, kernel thinning could in principle be used to reduce both the training set and the empirical-score sample, at the cost of inflating bias and variance by amounts that have not yet been characterized for score-debiased estimators. We view a careful theoretical and empirical study of kernel thinning under SD-KDE as out of scope for this paper but a promising follow-up.

C.3. Higher-order bias correction in KDE

The Laplace-corrected estimator of Section 5 belongs to the broader family of higher-order bias kernels. Classical work in this area includes the higher-order kernel constructions of Fan & Hu (1992) and the systematic comparison of Jones & Signorini (1997), both already discussed in the main related-work section. The novelty of our use of this construction is not in the kernel itself but in identifying it as the right first-order surrogate for SD-KDE in the limit where the empirical score is replaced by its leading-order Taylor expansion, and in giving a fused GPU realization that retains the same effective bias reduction at a fraction of the runtime cost.

C.4. KDE as a primitive in modern ML systems

Although the present paper targets density estimation directly, KDE-style sums also arise as computational primitives inside modern deep-learning pipelines. Self-attention with a softmax kernel can be written as a normalized weighted sum that is closely related to a Gaussian KDE evaluated at the query, and several recent works exploit this view to design sub-quadratic attention approximations. Zandieh et al. (2023) treat the attention denominator as a KDE and use a sub-quadratic KDE algorithm in the spirit of the FOCS line above to approximate it, obtaining provable spectral-norm guarantees and large end-to-end speedups. Kacham et al. (2024) replace the softmax kernel with a polynomial kernel and use polynomial-kernel sketches from randomized numerical linear algebra to obtain a linear-time attention mechanism with approximation guarantees. Streaming-style LSH sketches in the spirit of RACE (Coleman & Shrivastava, 2020) have similarly been adapted as the scoring primitive inside attention modules. FlashAttention (Dao et al., 2022) is the closest analogue on the hardware-aware side: it does not approximate the kernel sum but reorders the exact computation to fit the GPU memory hierarchy and to expose Tensor-Core GEMMs, which is the same high-level recipe we apply to SD-KDE.

These works underline that fast KDE-style primitives have downstream value beyond density estimation. The implementation pattern in this paper (matrix-multiplication reordering, streaming accumulation, mixed Tensor-Core/FP32 work) is a natural building block for any future architecture in which a score-corrected or higher-order kernel sum is used in place of vanilla attention or a vanilla KDE primitive.

C.5. Hybrid algorithmic–hardware directions for future work

The literature surveyed above suggests several hybrid directions that pair algorithmic reductions with the hardware-accelerated exact pipeline of this paper.

Approximate-score, exact-evaluation hybrids. The empirical score in SD-KDE is itself a sum of weighted kernels, and the SD-KDE estimator is empirically robust to small score perturbations. This makes it natural to plug a sub-quadratic approximate KDE algorithm, such as those of Charikar & Siminelakis (2017); Backurs et al. (2019); Siminelakis et al. (2019); Charikar & Siminelakis (2019); Charikar et al. (2020), into the score-evaluation step while retaining the exact Tensor-Core kernel of Section 4 for the final density evaluation. Quantifying how approximate-score error propagates into the bias-reduction guarantee of SD-KDE is an open theoretical question in this direction.

Spectral and grid-based score acceleration. For low-to-moderate dimensions, the score can also be computed by evaluating the gradient of a smoothed density on a grid, with non-uniform fast Fourier transforms (NUFFT) reducing the cost of the underlying sum. Combining a NUFFT-based score with the exact Flash-SD-KDE evaluation kernel is a promising hybrid for the dimensionalities we target, and is on our roadmap.

Coreset-then-accelerate. A complementary hybrid uses kernel thinning (Dwivedi & Mackey, 2021; Shetty et al., 2022) or another coreset construction to reduce n before invoking the exact pipeline, trading a controlled error increase for a square-root reduction in compute. Because Flash-SD-KDE is already efficient at large n, the regime where this trade is attractive likely lies at the very largest scales, where even a hardware-accelerated exact $O(n^2)$ pass becomes prohibitive.

In all three directions, a fast and accurate exact SD-KDE primitive is useful as the inner loop and as a ground-truth reference for evaluating approximation error, which is the role this paper aims to fill.